

Telecaller Backend (telebackend)

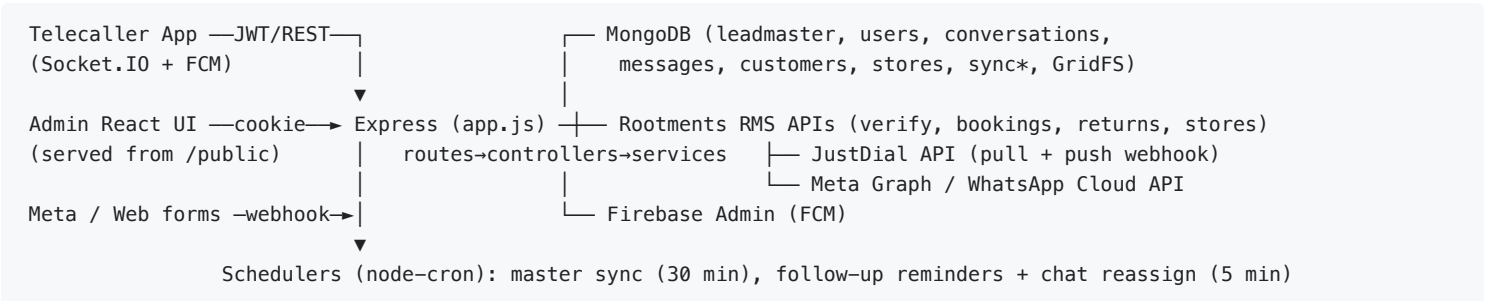
Backend, admin dashboard and omnichannel chat engine for a multi-brand **telecaller / CRM system**. Telecallers use a mobile app (JWT + Socket.IO + FCM push) to work leads and chat with customers; admins use a React dashboard served by the same server.

- **Runtime:** Node.js 20.x, Express 4, MongoDB (Mongoose 8)
- **Real-time:** Socket.IO 4 · **Push:** Firebase Cloud Messaging · **Scheduling:** node-cron
- **Integrations:** Rootments RMS (employee verification, bookings, returns, stores), JustDial, Meta (WhatsApp Cloud API, Instagram, Facebook Messenger)
- **Docs:** Swagger UI at /api-docs · **Hosting:** Render
- **Brands supported:** Zorucci, Suitor Guy, Dapper Squad (plus general fallback)

Table of Contents

1. [Architecture](#)
2. [Repository Layout](#)
3. [Getting Started](#)
4. [Configuration \(detailed\)](#)
5. [Startup Sequence](#)
6. [Authentication & Authorization](#)
7. [Data Models](#)
8. [API Reference](#)
9. [Lead Lifecycle & Status Rules](#)
10. [External Sync & Schedulers](#)
11. [Omnichannel Chat](#)
12. [Real-time Events \(Socket.IO\)](#)
13. [Push Notifications \(FCM\)](#)
14. [Admin Dashboard \(React\)](#)
15. [Utilities & Middleware](#)
16. [Scripts & Tests](#)
17. [Deployment](#)

1. Architecture



Layering: **routes** → **controllers** → **services** → **models**, with `utils/` helpers and `middlewares/` for auth/validation/errors. `server.js` wires HTTP + Socket.IO + schedulers around the Express `app` exported by `app.js`.

2. Repository Layout

Path	Purpose
server.js	Entry point: DB connect, HTTP server, Socket.IO, schedulers, graceful shutdown
app.js	Express app: middleware, route mounting, static admin UI, error handlers

Path	Purpose
package.json	Scripts (start , dev , sync), dependencies, Node 20 engine
.env.example	Template of all environment variables
serviceAccountKey.json	(local only) Firebase service account – never commit
src/config/	env.js (env + defaults), database.js (Mongo), firebase.js (FCM init), brandRegistry.js (channel→brand map)
src/routes/	15 route modules (see §8)
src/controllers/	Request handlers (leads, auth, chat, admin, webhooks, sync, ...)
src/services/	Business logic (chat, Meta send/profile, lead, customer, sync, notifications, sockets, GridFS)
src/models/	Mongoose schemas (§7)
src/schedulers/	masterSyncScheduler.js , followupReminderScheduler.js
src/middlewares/	authMiddleware , adminSession , validateRequest , errorHandler , notFound
src/utils/	ApiError , apiResponse , asyncHandler , phoneNormalizer , storeNormalizer , leadAssigner , mimeHelper , dateFilters , dateRange , pick
src/swagger/	swagger.js + one OpenAPI YAML per module
admin-frontend/	Vite + React 19 + Tailwind admin dashboard source
public/	Built admin UI (Vite builds here) + static assets
scripts/	Maintenance scripts and ad-hoc tests (§16)
sync/api/syncStores.js	Standalone store-sync runner
test-auth.js , curl_test.js	Manual helpers for verify-API and JSON-parse error checks

3. Getting Started

```
# 1. Install (Node 20.x)
npm install

# 2. Configure
cp .env.example .env      # then fill in values (see §4)

# 3. Run
npm run dev               # nodemon -r dotenv/config server.js
npm start                 # node server.js (production)
npm run sync              # manual sync: node scripts/runSync.js [all|returns|bookings|stores]

# Admin UI development (proxy /api → localhost:3000)
cd admin-frontend && npm install && npm run dev      # http://localhost:5173
npm run build                                         # outputs to ../public
```

URLs once running: API `http://localhost:3000/api` , health `/api/health` , Swagger `/api-docs` , admin UI `/` (redirects to `/admin/login` without a session cookie).

4. Configuration (detailed)

`dotenv` is loaded only when `NODE_ENV !== "production"` ; on Render, set variables in the dashboard. All values are read through `src/config/env.js` (`getEnv(key, fallback)`), which also supports **legacy alias names**.

4.1 Server & Database

Variable	Default (if any)	Description
PORT	3000	HTTP port
NODE_ENV	development	production disables .env loading

Variable	Default (if any)	Description
APP_BASE_URL	Render URL in <code>env.js</code>	Public base URL, used to build absolute media URLs for Meta
MONGODB_URI	<code>mongodb://localhost:27017/telecaller</code>	Mongo connection. If URI lacks <code>/telecaller</code> , <code>dbName: 'telecaller'</code> is forced (<code>database.js</code>)

4.2 Authentication & Admin

Variable	Default (if any)	Description
JWT_SECRET	–	Signs/verifies telecaller JWTs (7-day expiry) and Socket.IO auth. Required.
ADMIN_USERNAME / ADMIN_PASSWORD	–	Admin dashboard login. Required.
ADMIN_SESSION_TOKEN	–	Static token stored in <code>admin_session</code> cookie / <code>x-admin-token</code> header

4.3 Rootments (RMS) APIs

Variable (aliases)	Description
ROOTMENTS_API_TOKEN	Bearer token for the verify-employee API (required for telecaller login)
ROOTMENTS_VERIFY_API (<code>EXTERNAL_VERIFY_EMPLOYEE_URL</code>)	Employee verification endpoint
BOOKING_SUMMARY_URL (<code>BOOKING_CONFIRMATION_RMS_API_URL</code> , <code>RENTAL_BOOKING_SUMMARY_API</code>)	Booking summary API.
RETURN_REPORT_URL (<code>RETURN_RMS_API_URL</code> , <code>RENTAL_RETURN_REPORT_API</code>)	Return report API
STORE_LIST_API	Store list API

4.4 JustDial

Variable	Description
JUSTDIAL_API_URL	Pull API for JustDial leads
JUSTDIAL_SECRET	Token required on the JustDial push endpoint (<code>?token=<secret></code>)

4.5 Webhooks & Meta

Variable	Description
CUSTOM_WEBHOOK_API_KEY	Key for <code>/api/webhooks/lead-ingest</code> (<code>x-api-key</code> header or <code>apiKey</code> / <code>token</code> query)
META_WEBHOOK_VERIFY_TOKEN (<code>META_VERIFY_TOKEN</code>)	Meta webhook subscription token
META_APP_SECRET	Used for HMAC-SHA256 (<code>x-hub-signature-256</code>) signature checks on Meta payloads
META_ACCESS_TOKEN	Generic Meta Graph token (also used to discover Page tokens via <code>/me/accounts</code>)
WHATSAPP_PHONE_NUMBER_ID , <code>INSTAGRAM_ACCOUNT_ID</code>	Default (non-brand) channel IDs

4.6 Multi-brand channel IDs & tokens

Each brand (`ZORUCCI` , `SUITOR_GUY` , `DAPPER_SQUAD`) has its own set; this supports separate Meta Business portfolios.

Purpose	Variables (aliases)
WhatsApp phone number ID	<code>WA_PHONE_ID_<BRAND></code>
WhatsApp access token	<code>WA_ACCESS_TOKEN_<BRAND></code> (<code>WHATSAPP_ACCESS_TOKEN_<BRAND></code> , <code>WHATSAPP_TOKEN_<BRAND></code>)
Instagram business account ID	<code>IG_ACCOUNT_ID_<BRAND></code>
Facebook Page ID	<code>FB_PAGE_ID_<BRAND></code>
Facebook Page access token	<code>FB_PAGE_ACCESS_TOKEN_<BRAND></code> (<code>FB_PAGE_TOKEN_<BRAND></code> , <code>PAGE_ACCESS_TOKEN_<BRAND></code>); global fallback <code>FB_PAGE_ACCESS_TOKEN</code> / <code>PAGE_ACCESS_TOKEN</code> / <code>META_PAGE_ACCESS_TOKEN</code>

If a Page token is missing, `metaProfileService` tries to discover it from Meta (`/me/accounts`) and caches it.

4.7 Firebase (FCM)

Credential lookup order in `src/config/firebase.js`:

1. `FIREBASE_SERVICE_ACCOUNT_KEY` – raw JSON string **or** Base64 JSON
2. `FIREBASE_SERVICE_ACCOUNT_PATH` – absolute or relative to CWD
3. Default files: `serviceAccountKey.json`, `firebase-service-account.json`, `firebase-admin.json` (CWD or config dir)

If none is found a warning is logged and push notifications are disabled (the rest of the app works).

4.8 Brand registry (`src/config/brandRegistry.js`)

Maps an incoming Meta **channel ID** (WA phone ID / IG account ID / FB Page ID) to brand metadata `{brand, brandName, channel, storePrefix, themeColor}`. IDs are supplied via environment variables. Unknown IDs resolve to `general`.

Brand	storePrefix	Theme
Zorucci	Z-	#144234
Suitor Guy	SG-	#0B25B7
Dapper Squad	Dapper Squad-	#7B2869

4.9 Other config

- **CORS:** enabled with credentials.
- **JSON body:** `express.json` with `verify` hook storing `req.rawBody` for webhook signatures.
- **Swagger servers:** `http://localhost:3000/api` and relative `/api`; spec built from `src/swagger/*.yaml`.
- **Vite (`admin-frontend/vite.config.js`):** output `../public` (`emptyOutDir: false`), dev port 5173, `/api` proxy → `localhost:3000`.
- **Tailwind/PostCSS/ESLint:** `tailwind.config.js`, `postcss.config.js`, `eslint.config.js` in `admin-frontend/`.
- `.vscode/settings.json`: `git.ignoreLimitWarning: true`.

5. Startup Sequence (`server.js`)

1. Register `uncaughtException` / `unhandledRejection` loggers (process stays alive).
2. Load `.env` (non-production), `connectDB()`.
3. Create HTTP server → `socketService.initSocket(server)` (JWT-authenticated).
4. `listen(PORT)`.
5. `initializeMasterSyncScheduler()` – runs an initial sync if none has completed, then schedules incremental sync.
6. `initializeFollowupScheduler()` – schedules reminder/reassign sweep and runs one immediately.
7. `SIGTERM` → close server, exit 0.

6. Authentication & Authorization

Telecallers (`authMiddleware`)

Accepted credentials, in order:

1. **Authorization: Bearer <JWT>** – verified with `JWT_SECRET`; user loaded from DB (falls back to token claims if not found).
2. **x-user-id + x-password** headers – verified live against the Rootments API.
3. **Authorization: Basic base64(id:password)** – same verification.

Login (`POST /api/auth/telecaller-login`, `authController`): verifies via Rootments (`verifyTelecaller`, 20 s timeout, IPv4, retry on timeout/429/5xx), upserts the User (sets `isOnline`, `lastLoginAt`, optional `fcmToken`, removing that token from other users) and returns a 7-day JWT. `POST /api/auth/telecaller-logout` sets `isOnline=false`, `lastLogoutAt`.

Admin (`adminSession`)

`POST /api/admin/login` (or `/admin/login`) compares username (case-insensitive) and password with env config, then sets an `httpOnly`, `sameSite=lax`, 7-day `admin_session` cookie whose value is `ADMIN_SESSION_TOKEN`. `ensureAdminAuthenticated` accepts the cookie, `x-admin-token` header or Bearer token equal to that value; API callers get 401, browsers are redirected to `/admin/login`.

7. Data Models

Model (collection)	Key fields / notes
LeadMaster (leadmaster)	Single collection for all lead types. leadtype : booked, enquiry, bookingConfirmation, return, justdial, lossofsale. leadStatus : new, followup, complaint, completed. callStatus : connected, not connected, interested, not interested, forwarded, missed. Contact: phone , normalizedPhone (last 10 digits), customerName, brand, channel, store, source. Call data: callDuration, subCategory, closingReason/Action, itemCategory, remarks, functionDate. Flags: markasComplaint, markasFollowup, reminderSent. Follow-up: followupDate, followupclosingAction/remarks/callDuration. Booking/return: bookingNo, bookingDate, returnDate, deliveryDate, service, billReceived, amountMismatch, noofFunctions, noofAttires, competitor, rating, amounts, attendedBy. createdBy / updatedBy hold the assigned telecaller employeeId (or system). strict:false so flattened external API fields persist. Indexes: unique partial {bookingNo, leadtype} (bookingConfirmation/return), {normalizedPhone, leadStatus}, {leadStatus, updatedAt}, {store, leadStatus}, {followupDate, leadStatus, reminderSent}.
User (users)	employeeId (unique, uppercase), name , role (default Telecaller), store, phone, email, active, isOnline, lastLoginAt/LogoutAt, fcmToken, fcmTokenUpdatedAt.
Customer (customers)	Per-phone aggregate of lead state (latestLeadId , lead ID categories) maintained by customerService.recomputeCustomerState .
Conversation (conversati ons)	channel (whatsapp/instagram/facebook), brand, brandName, channelId, participant {phone, normalizedPhone, socialUserId, username, name, profilePic}, customerId, assignedTo, lastMessage, unreadCount, status (open/pending/resolved), leadId , lastActivityAt . Compound indexes for assignee/channel/brand queries.
Message (messages)	conversationId , messageId (unique sparse, Meta ID), channel, brand, senderType (customer/telecaller/system), messageType (text, image, audio, video, file, document, ig_reel, share, story_mention, fallback, template, interactive), text , caption , media fields (mediaFileId → GridFS omni_chat_media.files, mediaUrl, mimeType, fileName, isVoiceNote), status (sending/sent/delivered/read/failed), responseTimeSeconds, tempId, rawPayload, timestamp .
Store (stores)	externalId, locCode (unique), rawName, storeName, normalizedName, brand, location, status .
SyncLock (synclock)	Mutex document per jobName ; prevents overlapping syncs.
SyncLog (synclog)	Per-run summary (fetched/inserted/updated/skipped for booking & return).
SyncMeta (syncmeta)	Run history: type initial/incremental, trigger manual/auto/startup, status , per-job results . Drives "has initial sync happened?" logic.

8. API Reference

All routes are prefixed /api.  = authMiddleware (telecaller);  = admin session. Full schemas: [Swagger UI /api-docs](#) .

Auth & users

Method & path	Auth	Description
POST /auth/login	–	Generic employee login (JWT)
POST /auth/telecaller-login	–	Telecaller login, upserts user, returns JWT
POST /auth/telecaller-logout		Mark offline
POST/DELETE /users/fcm-token (aliases /user/fcm-token , /auth/fcm-token)		Register / remove FCM token
GET /users/fcm-token/status		Token registration status
POST /users/test-notification		Send a test push

Leads (all)

Method & path	Description
POST /leads	Create manual lead
GET /leads/completed, GET /leads/performance	Completed leads; own performance stats
GET /leads/enquiries (/enquiry), GET/POST /leads/enquiries/:id	Enquiry leads – list/get/update
GET /leads/lossofsale (/loss-of-sale), GET/POST .../:id	Loss-of-sale leads
GET /leads/booked, GET/POST /leads/booked/:id	Booked leads

Method & path	Description
GET/POST /leads/:id	Generic get/update (24-hex ObjectId)
GET /leads/followups, GET/POST /leads/followups/:id	Follow-ups
GET /leads/complaints, GET/POST /leads/complaints/:id	Complaints
GET /leads/booking-confirmation, GET/POST .../:id	Booking-confirmation calls
GET /leads/returns, GET/POST /leads/returns/:id	Return-feedback calls
GET /leads/justdial, GET /leads/justdial/:id, POST .../:id	JustDial leads
GET /customers/check-phone, GET /customers/:id/history	Customer lookup & history
GET /stores	Store list
POST /sync/stores, /sync/returns, /sync/booking-confirmation(s)	Manual syncs

Chat (/api/chat)

Method & path	Auth	Description
GET /chat/media/:fileId	public	Stream media from GridFS (range supported)
GET /chat/conversations, GET .../:id, GET .../:id/messages		List / detail / history
POST /chat/conversations/:id/messages		Send text/media/template message
POST /chat/conversations/:id/media		Upload file (multer , field file) and send
POST /chat/conversations/:id/read		Mark as read
POST /chat/conversations/:id/convert-lead		Create lead from chat
POST /chat/conversations/:id/transfer		Reassign to another telecaller
POST /chat/send-brochure-template		Send WhatsApp brochure template
POST /chat/simulate-inbound		Test inbound without Meta

Webhooks (public, own verification)

Method & path	Description
GET /webhooks/meta	Meta subscription verification (hub.* params)
POST /webhooks/meta	Inbound WhatsApp (whatsapp_business_account), Instagram (instagram), Facebook (page). Replies 200 EVENT_RECEIVED immediately, then processes
POST /webhooks/lead-ingest	External web/ads leads, protected by CUSTOM_WEBHOOK_API_KEY
GET/POST /justdial/lead	JustDial push; always returns RECEIVED

Admin

Method & path	Auth	Description
POST /admin/login, POST /admin/logout	–	Admin session
GET /admin/dashboard-summary, /admin/telecaller-leaderboard		Dashboard KPIs, leaderboard
GET /admin/telecallers/:employeeId/summary · /category-performance · /recent-calls		Telecaller drill-down
PUT /admin/telecallers/:employeeId (/profile)		Edit telecaller
GET /admin/reports/completed-leads (+ /export)		Call reports / export
GET /admin/chat-reports/summary, /telecaller-performance, /conversations, /export		Chat analytics
GET /admin/dashboard, /telecaller-summary, /reports, /complaints/pivot, /filter-options		Legacy admin API (adminController)

System

GET /api/health (uptime, memory, DB state, Node version, PID; /health redirects here), GET /api-docs (Swagger).

Route order matters in `app.js`: new admin routes and legacy admin routes are mounted **before** `leadRoutes`, whose router-level `authMiddleware` would otherwise intercept them. Unknown `/api/*` → `notFound` (404 JSON); everything else → `errorHandler`.

9. Lead Lifecycle & Status Rules

Implemented in `statusResolverService.js`. Explicit flags always win.

Lead kind	Rule
Manual / enquiry / JustDial	<code>markasComplaint</code> → complaint ; <code>markasFollowup</code> → followup ; else by <code>callStatus</code> : connected, not interested, forwarded → completed ; not connected, interested → followup ; otherwise new
Booking confirmation	<code>complaint flag</code> → complaint ; <code>followup flag</code> → followup ; <code>billReceived = no</code> or <code>amountMismatch</code> → complaint ; else completed
Return	<code>complaint flag</code> → complaint ; <code>followup flag</code> → followup ; else completed

Follow-ups set `followupDate`; the scheduler later alerts the owner once (`reminderSent`). Synced leads are auto-distributed by `utils/leadAssigner.js` (round-robin across telecallers with `isOnline` and `lastLoginAt` within 12 h, excluding role `admin`), setting `createdBy` / `updatedBy` and sending a "New Leads Assigned" push.

10. External Sync & Schedulers

Master sync (`masterSyncScheduler.js`)

- **Startup:** if no `SyncMeta` initial sync with status `completed/partial` exists → run **initial** (60-day lookback), non-blocking.
- **Cron** `* / 30 * * * *`: **incremental** sync (7-day lookback).
- **Locking:** `SyncLock` document prevents overlap; skipped if locked.
- **Steps (each isolated, failures don't stop the rest):** stores (`storeSyncService`) → booking confirmations (`syncBookingConfirmationLeads`) → returns (`syncReturnLeads`) → JustDial (`justDialSync`).
- **Result:** `completed` (all ok), `partial` (some failed), `failed` (all failed), stored in `SyncMeta` / `SyncLog`.
- Sync services POST `{dateFrom, dateTo}` to the RMS (15-min timeout), skip bookings already **completed**, upsert by `{bookingNo, leadtype}`, normalize phone/store, then recompute `Customer` state and assign new leads.
- JustDial pull dedupes by `normalizedPhone`: existing phone → type updated to `justdial` keeping status; else new lead (`source: justDialSync`).

Follow-up & chat scheduler (`followupReminderScheduler.js`)

Every **5 min** (and once on boot):

1. Find `≤500` leads with `leadStatus=followup`, `followupDate ≤ now`, `reminderSent != true`; emit `followup:reminder` over Socket.IO + FCM to `updatedBy`; set `reminderSent=true`.
2. `chatService.reassignPendingSystemChats()` – reassigns open chats owned by `system` /unassigned to online telecallers.

11. Omnichannel Chat

Core in `services/chatService.js` (+ `metaSendService`, `metaProfileService`, `gridfsService`, `mimeHelper`).

Inbound (`processInboundWhatsApp/Instagram/Facebook`): resolve brand via `brandRegistry`; find/create `Conversation` (per participant & channel, guarded by in-memory `withParticipantLock` to avoid duplicates); dedupe on `messageId`; download media from Meta/CDN → store in **GridFS bucket** `omni_chat_media` with MIME detection (magic bytes); resolve IG/FB profile (name, username, picture; cached with TTL); compute telecaller **response time**; link `Customer`; assign telecaller; emit sockets/push.

Assignment (`findOrAssignTelecaller`): (1) keep continuity with the telecaller owning the customer's latest active lead; (2) otherwise least-loaded (fewest open chats) among online telecallers (12 h login window); (3) else `system`, later fixed by the 5-min sweep.

Outbound (`sendOutboundMessage`): persists message (optimistic `tempId`), sends through WhatsApp Cloud API / Instagram / Messenger Graph APIs using brand-specific tokens, supports text, media (public URL built from `APP_BASE_URL`), templates (brochure), read receipts; status moves `sending` → `sent/failed` and is updated by delivery webhooks (`delivered` , `read`).

Other operations: `markConversationAsRead` , `convertChatToLead` , `transferConversation` , `simulateInboundMessage` (testing).

12. Real-time Events (Socket.IO)

Connection requires a valid telecaller JWT; each socket joins room `room:telecaller_<EMPLOYEEID>` .

Event	Trigger
<code>chat:new_message</code>	Inbound/outbound message stored
<code>chat:assigned</code>	Conversation (re)assigned to the telecaller
<code>chat:status_update</code>	Message delivery/read/fail updates
<code>chat:conversation_updated</code>	Conversation metadata changed
<code>lead:new</code>	New lead assigned (e.g. via lead-ingest)
<code>followup:reminder</code>	Follow-up due

13. Push Notifications (FCM)

`notificationService` : `sendDirectNotification` , `sendNotificationToUser(employeeId)` , `sendNotificationToUsers` . Data payload values are stringified (FCM requirement). Used for new-lead batches (`lead_batch`), follow-ups (`followup`), and chat events. Tokens stored on `User.fcmToken` ; invalid tokens are handled in the service.

14. Admin Dashboard (React)

`admin-frontend/` — Vite 8, React 19, React Router 7, Tailwind 3, Recharts, lucide-react, axios (`withCredentials`).

Routes: `/admin/login` , `/admin/dashboard` , `/admin/telecallers` , `/admin/telecallers/:employeeId` , `/admin/call-reports` , chat reports page. Components: `Layout` , `DialexLogo` , `EditTelecallerModal` . Build output goes to `public/` , served by Express with SPA fallback (non- `/api` GETs → `public/index.html` , redirecting to `/admin/login` when no `admin_session` cookie).

15. Utilities & Middleware

- **ApiError** / **apiResponse** / **asyncHandler** / **errorHandler** / **notFound** – uniform `{success, message, data}` JSON errors.
- **validateRequest** – express-validator result handler.
- **phoneNormalizer** – strips to last 10 digits (`normalizedPhone`).
- **storeNormalizer** – canonical store names + `buildStoreRegex` for flexible search.
- **mimeHelper** – extension & magic-byte MIME detection; normalizes streaming Content-Type.
- **dateFilters** / **dateRange** / **pick** – query-date handling and object whitelisting.

16. Scripts & Tests

No test framework; scripts run with plain `node` .

Script	Purpose
<code>scripts/runSync.js</code> [all returns bookings stores]	Manual sync (60-day)
<code>scripts/backfill-profiles.js</code>	Backfill IG/FB participant profiles
<code>scripts/merge-duplicate-conversations.js</code>	Merge duplicate conversations/messages
<code>scripts/test-meta-profile.js</code> , <code>test-message-enum.js</code> , <code>test-media-normalization.js</code>	Unit-style checks
<code>scripts/test-omni-media-gridfs.js</code> , <code>test-http-media-endpoints.js</code>	GridFS/media integration checks (need DB)

Script	Purpose
test-auth.js	Calls the Rootments verify API with env config
sync/api/syncStores.js	Standalone store sync

17. Deployment

Designed for **Render** (Node 20): set `NODE_ENV=production` and all variables from §4 in the dashboard; build admin UI (`cd admin-frontend && npm run build`) and commit `public/` . Start command `npm start` . Configure Meta webhook to `https://<host>/api/webhooks/meta` with the verify token, and the JustDial push URL to `https://<host>/api/justdial/lead?token=<JUSTDIAL_SECRET>` . Branches in git: `main` (production) plus feature branches.